

Clase del Módulo 1

Sitio: Campus - Sadosky Capacitaciones
Curso: Introducción al mundo de la programación paralela para HPC_V1
Libro: Clase del Módulo 1

Imprimido por: Pablo Carrai
Día: martes, 2 de junio de 2026, 15:29

Descripción

Computación Paralela para resolver problemas enormes

Tabla de contenidos

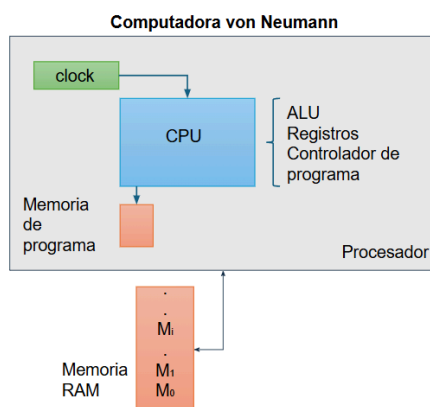
1. **¿Qué es el paralelismo y para qué lo necesitamos?**
¿Por qué necesitamos el paralelismo?
2. **Conceptos básicos**
3. **Paralelismo vs. concurrencia**
4. **Arquitecturas para programación paralela (CPU multicore, GPU, clústeres)**
 - 4.1 Modelo de memoria compartida: CPU multicore
 - 4.2 Modelo de memoria compartida: GPU
 - 4.3 Clústeres de computadoras
5. **Terminología de Programación Paralela**
6. **Cierre: diferentes caminos hacia un mismo objetivo**

1. ¿Qué es el paralelismo y para qué lo necesitamos?

El espectacular crecimiento del rendimiento computacional al que nos acostumbramos durante décadas ya no proviene del aumento vertiginoso en la velocidad de un único procesador. La era donde cada nueva generación de hardware aceleraba el software existente sin esfuerzo por parte del programador, ha terminado. Hoy, el camino hacia una mayor potencia de cálculo reside en la utilización eficiente de múltiples núcleos de procesamiento.



Esta sección analiza por qué este cambio de paradigma es fundamental para la computación moderna.



El modelo de computación tradicional, conocido como la arquitectura de von Neumann, es inherentemente secuencial: ejecuta una única instrucción a la vez sobre un único flujo de datos.

Si bien este modelo ha servido como la base de la computación durante más de medio siglo, sus limitaciones físicas y energéticas han frenado el escalado de la frecuencia de reloj. Para continuar avanzando, es necesario adoptar un enfoque diferente.

La computación paralela emerge como la solución a este estancamiento. Se define como el uso simultáneo de múltiples recursos de cómputo para resolver un problema. En lugar de ejecutar una larga secuencia de instrucciones una tras otra, dividimos el problema en partes discretas que pueden resolverse concurrentemente. La computación paralela es la evolución de la computación secuencial que intenta emular lo que sucede en el mundo real, donde varios eventos complejos e interrelacionados suceden al mismo tiempo.

En otras palabras, la computación paralela se basa en la premisa de que un problema puede dividirse en subproblemas más pequeños que pueden resolverse de manera simultánea. En su esencia, implica la coordinación y sincronización de múltiples unidades de procesamiento que trabajan sobre diferentes partes de una misma tarea. El objetivo final es reducir el tiempo total de ejecución y aumentar la eficiencia en el uso de recursos computacionales.

¿Por qué necesitamos el paralelismo?

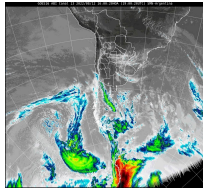
¿Por qué necesitamos el paralelismo?

Las razones principales para adoptar el paralelismo son claras y contundentes:

- **Ahorrar tiempo:** Múltiples procesadores trabajando en conjunto pueden resolver un problema en una fracción del tiempo que le tomaría a uno solo.
- **Resolver problemas de gran escala:** Ciertas aplicaciones son tan masivas en sus requerimientos de cálculo que resultan inviables en un sistema secuencial, sin importar cuánto tiempo se espere.

Existen también razones secundarias, como el **ahorro de costos** al construir clústeres con hardware económico o el **aprovechamiento de recursos de cómputo remotos** a través de redes.

Un ejemplo clásico que ilustra la necesidad indispensable del paralelismo es la **predicción del tiempo**.



Consideremos la tarea de **modelar el clima** sobre una superficie de 20 millones de km² y una altitud de 20 km. Para predecir el estado del tiempo con solo una hora de antelación, se requieren aproximadamente **4×10^{13} cálculos**. Si quisiéramos una predicción para las próximas 48 horas, la cantidad de operaciones ascendería a casi **2×10^{15}** . Un procesador secuencial capaz de ejecutar mil millones de cálculos por segundo tardaría días en completar esta tarea, un tiempo inútil para una predicción meteorológica. Sin embargo, distribuyendo el trabajo entre **miles de procesadores**, esta misma tarea puede completarse en cuestión de minutos.

Como podemos ver en el ejemplo, la computación paralela nos permite resolver **problemas enormes**. En el siguiente capítulo nos adentraremos en los conceptos básicos que nos permitirán introducirnos en el tema.

2. Conceptos básicos

En una aplicación paralela, el trabajo total se reparte entre distintas unidades de procesamiento (como núcleos de CPU, GPU o nodos de un clúster). Cada una ejecuta un subconjunto de instrucciones, y los resultados parciales se combinan al final para obtener la solución global. El grado de paralelismo depende de **tres factores**:

> El tipo de problema

> La granularidad de las tareas

> El modelo de memoria



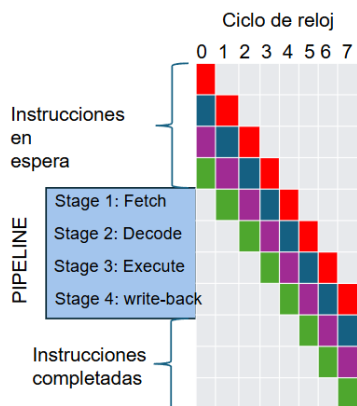
A su vez, el paralelismo puede manifestarse en distintos niveles de la arquitectura computacional:

- a nivel de instrucción (*Instruction-Level Parallelism* o ILP)
- a nivel de datos (*Data-Level Parallelism* o DLP)
- a nivel de tarea (*Thread-Level Parallelism* o TLP).

Veamos cada una de ellas en detalle.

Paralelismo a nivel de instrucción (ILP)

Optimizaciones internas del procesador que permiten ejecutar varias instrucciones simultáneamente (*pipelining*).



Por ejemplo, mientras una instrucción se está buscando (F), otra puede estar decodificándose (D), una tercera ejecutándose (E) y otra en etapa de escritura (W).

En el gráfico de la izquierda podemos visualizar estas acciones en el recuadro celeste como *Fetch*, *Decode*, *Execute* y *Write-back*.

Paralelismo a nivel de datos (DLP)

La misma operación se aplica sobre múltiples elementos o datos. Cuando se utilizan datos de 1 dimensión, estos se pueden distribuir en bloques o en forma cíclica (*Roud-Robin*), como muestra la figura 1. En la figura 2, podemos observar también distribución en 2 dimensiones.

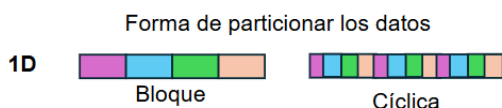


Figura 1

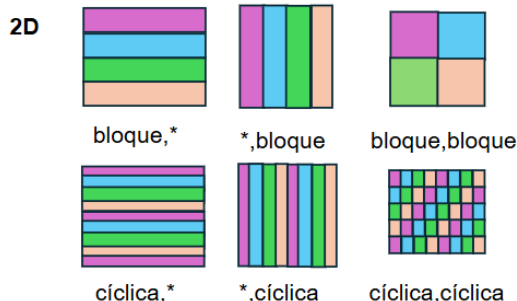


Figura 2

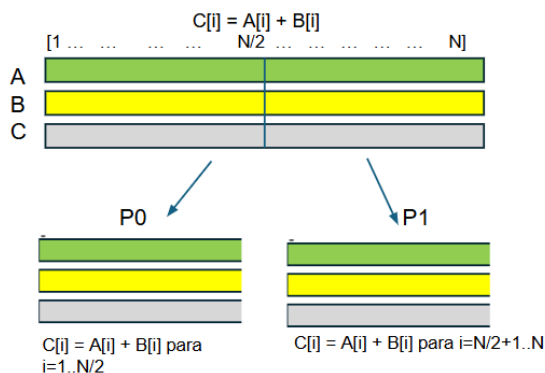


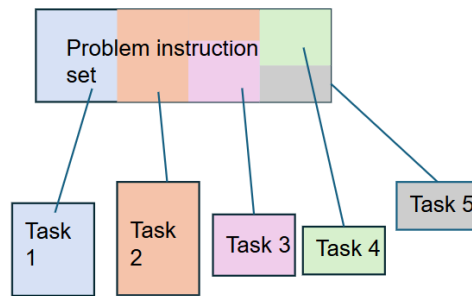
Figura 3

La figura 3 muestra el paralelismo a nivel de datos para los arreglos de 1 dimensión A, B y C, usando dos procesos P0 y P1.

Paralelismo a nivel de tarea (TLP)

En este caso, distintas tareas o procesos se ejecutan en paralelo, cada uno con un flujo de control independiente.

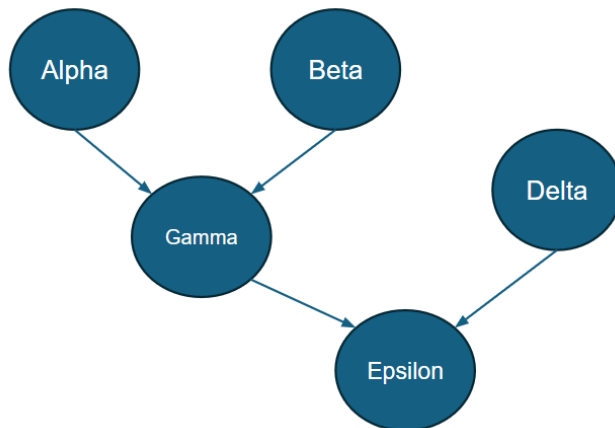
Se enfoca en la división de tareas



El siguiente ejemplo muestra cómo se puede ejecutar paralelismo a nivel de tareas:

```
v = alpha();
w = beta();
x = gamma(v, w);
y = delta();
printf ("%6.2f\n", epsilon(x,y));
```

A continuación, en el grafo se muestra la dependencia de las tareas. Se pueden ejecutar alpha y beta en paralelo. Luego gamma y delta en paralelo. Y finalmente, epsilon.



3. Paralelismo vs. concurrencia

En el ámbito de la computación moderna, los términos paralelismo y concurrencia suelen utilizarse de manera indistinta, aunque en realidad describen conceptos complementarios pero diferentes. Comprender la diferencia entre ambos es fundamental para diseñar software eficiente que aproveche al máximo los recursos disponibles.

Definiciones básicas

Paralelismo



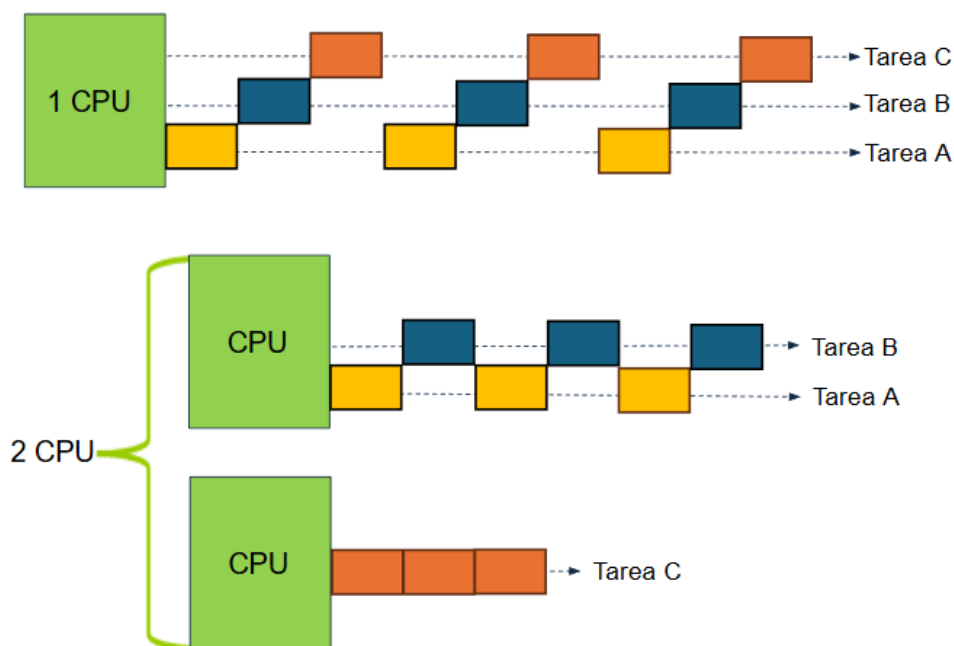
Se refiere a la ejecución simultánea de múltiples operaciones. Es un concepto físico y está relacionado directamente con la utilización simultánea de varios recursos de hardware (múltiples núcleos, procesadores o máquinas). Un sistema es paralelo cuando dos o más operaciones se ejecutan literalmente al mismo tiempo. Ejemplo: procesar distintos fragmentos de una imagen en diferentes núcleos de CPU (*Central Processing Unit*) o GPU (*Graphics Processing Unit*) al mismo tiempo.

Concurrencia



Describe la capacidad de un sistema para manejar múltiples tareas en progreso, pero no necesariamente de manera simultánea. Se centra en la estructura del software y en cómo se organizan las tareas que pueden intercalarse o superponerse en el tiempo. Cuando tenemos un solo procesador se produce un intercalado de las instrucciones de ambos procesos, de tal forma que tenemos la sensación de que hay un paralelismo en el sistema. Por ejemplo, un servidor web que atiende múltiples solicitudes de usuarios casi al mismo tiempo mediante *threads* o eventos, aunque internamente solo un núcleo las ejecute de forma secuencial y alternada.

Cuando hay más tareas que procesadores, las tareas se multiplexan en el tiempo:



En síntesis, la concurrencia trata sobre la composición de tareas independientes, mientras que el paralelismo trata sobre su ejecución simultánea. Una frase muy utilizada para resumirlo es:

“La concurrencia es sobre hacer que parezca que muchas cosas suceden a la vez; el paralelismo es hacer que muchas cosas sucedan a la vez”.



Veamos a continuación un video que explica esta diferenciación de manera gráfica:

Concurrencia Vs Paralelismo en procesos

INGEN TC Academy

Ver en

Fuente: INGEN TC Academy. (2024, 12 de agosto). Concurrencia Vs Paralelismo en procesos [Video]. YouTube.

Veamos un ejemplo. Consideremos un programa que debe realizar tres operaciones: leer datos de un archivo, procesarlos y guardarlos en una base de datos.

Enfoque concurrente	Enfoque paralelo
El programa podría iniciar la lectura de un archivo mientras espera que la base de datos responda, aprovechando el tiempo de espera de una tarea para avanzar en otra. No todas las operaciones se ejecutan al mismo tiempo, pero el sistema mantiene múltiples tareas “vivas” y las alterna eficientemente.	En este enfoque, cada operación se ejecutaría en un núcleo diferente del procesador, ocurriendo todas simultáneamente.

Ambas estrategias pueden coexistir: un sistema puede ser concurrente en su estructura y paralelo en su ejecución.

Sincronización y comunicación

Tanto en sistemas concurrentes como paralelos, el control de acceso a los recursos compartidos es un aspecto crítico. Para evitar **problemas de carrera**, los mecanismos típicos incluyen:

> Semáforos, mutexes y monitores
> Colas y buffers de mensajes
> Barreras y sincronizadores



Se produce un **problema de carrera** cuando dos o más *threads* (hilos) o procesos acceden simultáneamente a un recurso compartido y al menos uno de ellos lo modifica. El resultado final depende del orden de ejecución.

Un diseño inadecuado de sincronización puede derivar en condiciones de carrera, interbloqueos (*deadlocks*) o esperas activas, reduciendo drásticamente la eficiencia del sistema.

Complementarios, no opuestos

La concurrencia y el paralelismo no son opuestos, sino niveles complementarios de diseño.



Un sistema de alto desempeño —como los diseñados en entornos de HPC— requiere comprender y combinar ambos conceptos de forma equilibrada, adaptándolos al hardware y a los objetivos del problema.

4. Arquitecturas para programación paralela (CPU multicore, GPU, clústeres)

La forma en que las arquitecturas paralelas organizan la memoria es un factor determinante en el diseño del software. Define cómo las tareas comparten información, cómo se sincronizan y qué modelos de programación son más naturales para un sistema determinado. Fundamentalmente, existen dos enfoques principales: la memoria compartida y la memoria distribuida.



En el modelo de **memoria compartida**, todos los procesadores están conectados a un único espacio de memoria global y pueden acceder a cualquier dirección de esta. El cambio de un dato en memoria por parte de un procesador es visible inmediatamente para todos los demás. La comunicación es implícita: las tareas simplemente leen y escriben en variables compartidas.



En el modelo de **memoria distribuida**, cada procesador tiene su propia memoria privada. No existe un espacio de direcciones global; la memoria de un procesador no es directamente accesible por otro. Para compartir datos, las tareas deben comunicarse de forma explícita a través de una red de interconexión, un proceso conocido como **pasaje de mensajes**. Estos sistemas se interconectan mediante una red, que puede ser tan simple como un *bus* (poco escalable) o tan compleja como una jerarquía de *switches* (ej.: *fat-tree*) para sistemas masivos.



Existe un tercer enfoque denominado **modelos híbridos** que combinan memoria compartida dentro de cada nodo y paso de mensajes entre nodos (muy usados en supercomputadoras).

A continuación profundizaremos en los modelos en cada una de las arquitecturas paralelas.

4.1 Modelo de memoria compartida: CPU multicore

4.1 Modelo de memoria compartida: CPU multicore

Las CPU *multicore* –se basan en el modelo de memoria compartida– son procesadores que integran varios núcleos de ejecución o *cores* independientes dentro de un mismo chip. Cada núcleo posee su propia unidad de control y de procesamiento aritmético-lógico (ALU, *arithmetic and logic unit*), lo que permite ejecutar múltiples hilos o procesos en paralelo.

Sus características principales son:

- > Memoria compartida
- > Baja latencia de comunicación
- > Paralelismo moderado
- > Programación

A su vez, podemos mencionar ventajas y desafíos:



- Alta compatibilidad con software existente.
- Bajo consumo energético comparado con arquitecturas distribuidas.
- Ideal para servidores, estaciones de trabajo y aplicaciones multithread (simulaciones, bases de datos, videojuegos).
- Limitada escalabilidad: el aumento de núcleos no siempre implica un aumento lineal del rendimiento.
- Sincronización y coherencia de caché se vuelven más complejas a medida que crece el número de threads.

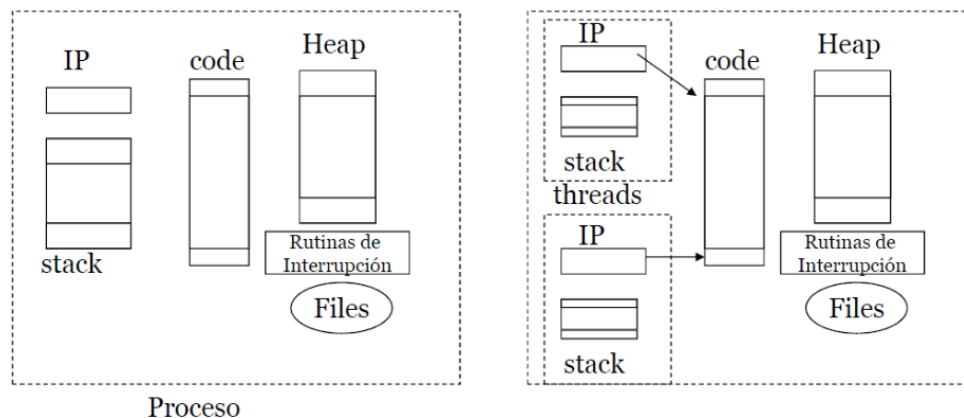
Hilos o *threads*

Los procesos tienen componentes del estado de ejecución de un programa y regiones o mecanismos asociados al manejo de memoria y control del procesador:

- El **IP (*instruction pointer*)**, que es un registro del procesador que contiene la dirección de memoria de la próxima instrucción que se va a ejecutar;
- el ***stack* o pila** es una estructura de memoria utilizada para gestionar llamadas a funciones;
- el ***code*** que es el segmento de código es la región de memoria donde se almacenan las instrucciones ejecutables del programa;
- el ***heap*** que es una región de memoria usada para la asignación dinámica de memoria durante la ejecución del programa;
- las **rutinas de interrupción** que son funciones especiales que ejecuta el sistema cuando ocurre una interrupción;
- y los ***files* o archivos** son estructuras de almacenamiento persistente gestionadas por el sistema operativo.

Por su parte, los **hilos o *threads*** son rutinas livianas que comparten la misma zona de memoria y las variables globales. Crear un proceso implica duplicar estructuras del sistema operativo (tablas de páginas, descriptors de archivos, etc.), pero crear un hilo solo requiere asignar su pila y contexto, ya que comparte el resto de recursos con el proceso. En la práctica, el tiempo de creación de un hilo puede ser entre 100 y 1000 veces menor que crear un proceso ($\approx 2-3$ órdenes de magnitud), dependiendo del sistema operativo.

Veamos un esquema de cómo funcionan estos elementos:



Cada hilo tiene su propio identificador y encapsula su propia ruta de ejecución. A su vez, cada uno tiene estados locales y recursos compartidos a través de los cuales se comunican, como la memoria.

4.2 Modelo de memoria compartida: GPU

4.2 Modelo de memoria compartida: GPU

Las GPU (por las siglas en inglés, *Graphics Processing Unit*) se basan en el modelo de memoria compartida. Surgieron originalmente para el procesamiento gráfico, pero su arquitectura altamente paralela las ha convertido en un componente esencial de la HPC. Las GPU son aceleradores de hardware que pueden utilizarse para transferir tareas de cómputo desde las CPU.

A diferencia de las CPU, las GPU están optimizadas para ejecutar miles de threads ligeros en paralelo, aplicando la misma operación sobre grandes conjuntos de datos (paralelismo masivo a nivel de datos). Son adecuadas para cálculos regulares que operan sobre vectores o matrices de datos estáticos.

Sus principales características son:

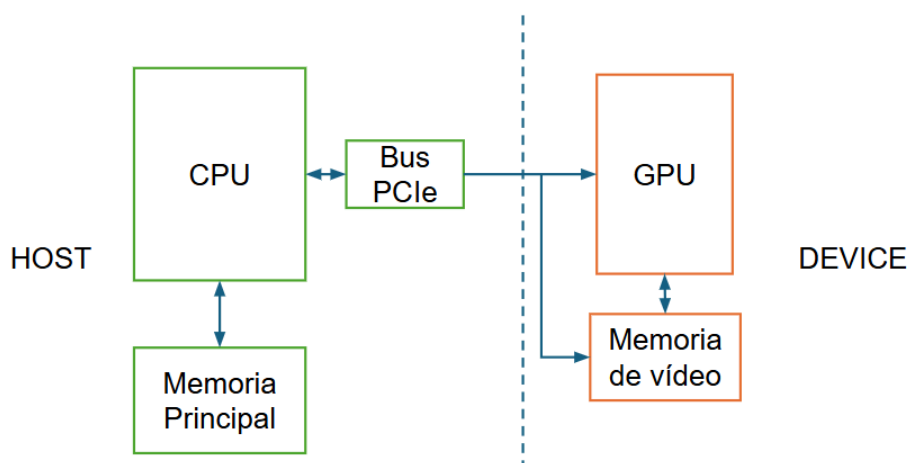
- > Miles de núcleos simples
- > Memoria jerárquica especializada
- > Programación

Este modelo también tiene sus propias **ventajas y desafíos**.

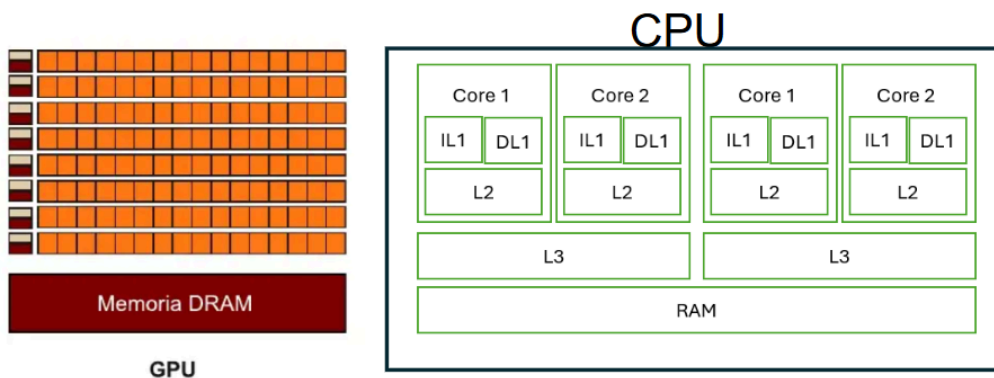


- Altísima capacidad de cómputo para tareas vectoriales o matriciales.
- Ideal para aplicaciones con paralelismo masivo: aprendizaje profundo, procesamiento de imágenes, simulaciones físicas, etc.
- Excelente relación costo/rendimiento para operaciones de punto flotante.
- Programación más compleja que en CPU.
- Transferencia de datos entre CPU y GPU (bus PCIe) puede convertirse en cuello de botella.
- Menor flexibilidad en tareas con dependencias o flujos de control complejos.

Las arquitecturas utilizadas para programar las GPUs se pueden dividir en dos partes. El HOST contiene el procesador (CPU) conectado a la memoria principal y al puerto PCI express, que a su vez se conecta a la GPU. A la tarjeta gráfica se la llama DEVICE, la cual tiene una memoria (*device memory*) y está formada por varios multiprocesadores.



La GPU se construye sobre un arreglo de *Streaming Multiprocessors* (SM), formado por *cores*. Cada *core* está diseñado para llevar a cabo operaciones matemáticas, o bien, de direccionamiento de datos en memoria y la transferencia posterior. Además, cada SM dispone de una memoria caché compartida, accesible por todos los núcleos que componen la GPU, lo que permite acelerar el intercambio de datos y reducir la latencia de acceso a memoria global.

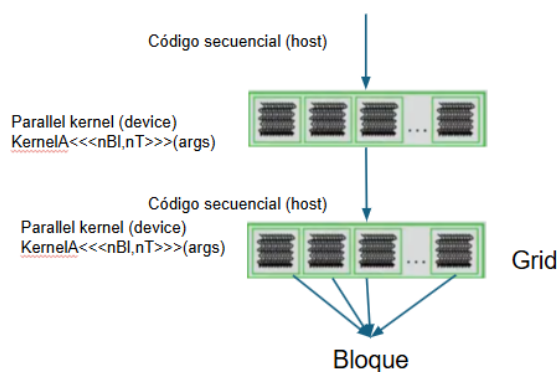


Fuente: Guim F. & Rodero I. (2013) Arquitecturas basadas en computación gráfica (GPU). UOC.

Un programa multiproceso se divide en bloques de hilos que se ejecutan independientemente unos de otros, de modo que una GPU con más multiprocesadores ejecutará automáticamente el programa en menos tiempo que una GPU con menos multiprocesadores.

El código secuencial se ejecuta en la CPU o *host* y el código paralelo -llamado *kernel*- en la GPU o *device*.

Un **kernel** es una función ejecutada por un conjunto de hilos, donde todos los hilos ejecutan el mismo código, aunque operan sobre diferentes datos.



4.3 Clústeres de computadoras

4.3 Clústeres de computadoras

Un **clúster** es un conjunto de computadoras (nodos) conectadas por una red de alta velocidad, que cooperan para resolver un mismo problema como si fueran un solo sistema. Se basan en el modelo de **memoria distribuida**.

Cada nodo puede tener una o varias CPU y/o GPU, combinando así diferentes niveles de paralelismo.

Sus principales características son:

- > Memoria distribuida
- > Alta escalabilidad
- > Comunicación
- > Gestión de tareas

Al igual que en los apartados anteriores, podemos repasar **ventajas y desafíos** en este modelo:



- Permite alcanzar potencias de cómputo imposibles para un solo equipo.
- Escalable y adaptable: se pueden agregar nodos para aumentar el rendimiento.
- Base de la mayoría de los sistemas HPC y supercomputadoras modernas.
- Comunicación entre nodos más lenta que entre núcleos de una CPU.
- Mayor complejidad de programación y depuración (sincronización, partición de datos, fallos).
- Requiere infraestructura especializada (red de interconexión, refrigeración, energía).

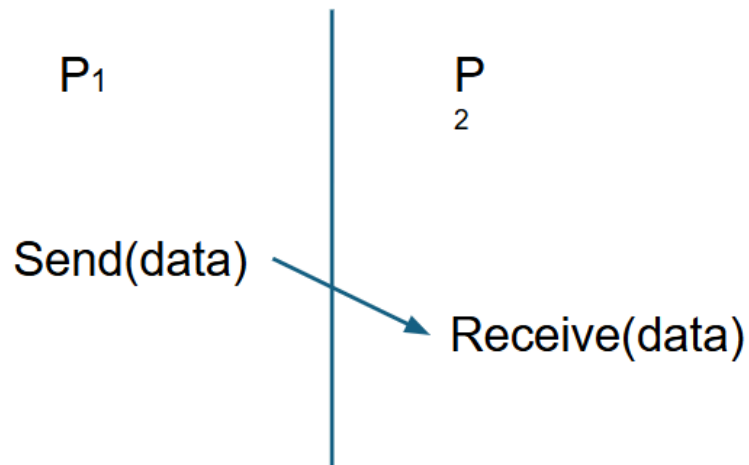
Veamos ahora un **ejemplo**:

Comparemos la multiplicación de un vector por una matriz en un procesador y en tres procesadores:

Utilizando un procesador	Utilizando tres procesadores
0 1 0 x 1 4 7 2 5 8 3 6 9 2 5 8	0 1 0 x 1 4 7 2 5 8 3 6 9 0*1+1*2+0*3 1*4+1*5+0*6 0*7+1*8+0*0 Proc0 Proc1 Proc2
Si calcular cada resultado toma 5 unidades de tiempo, el tiempo total es 15.	El tiempo total es el máximo del tiempo de ejecución en P0, P1 y P2. Por lo tanto el tiempo se reduce a 5 unidades.

Comunicación básica

Las arquitecturas con memoria distribuida usan comunicación explícita para intercambiar datos. Los programadores deben insertar en su código rutinas específicas de envío (*send*) y recepción (*receive*) para intercambiar datos. El intercambio de datos requiere sincronización (cooperación) entre los *senders-receivers*.



La comunicación mediante pasaje de mensajes se puede clasificar según dos características ortogonales:

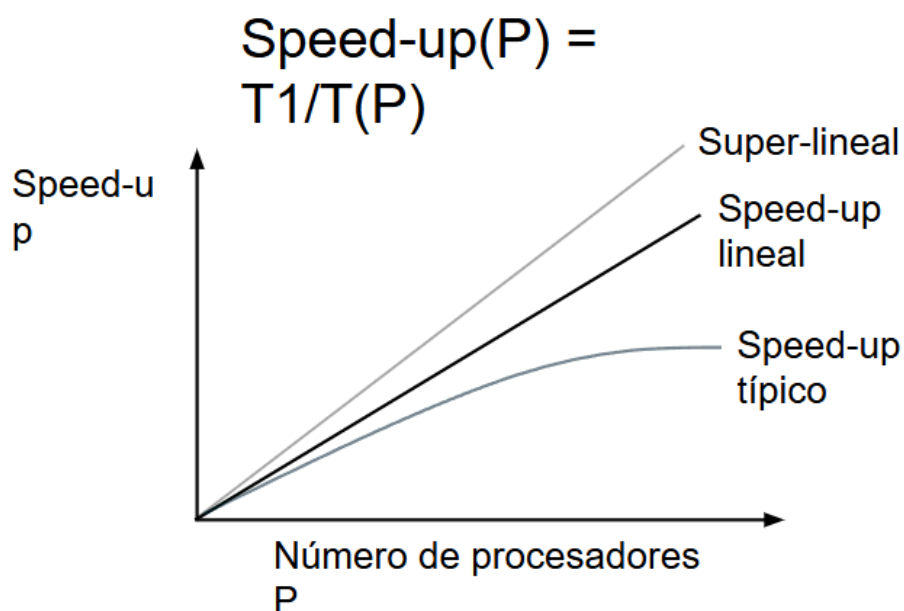


5. Terminología de Programación Paralela

Para discutir, diseñar y evaluar sistemas paralelos de manera efectiva, es crucial dominar un vocabulario común. La terminología correcta nos permite describir con precisión la estructura de un programa, los desafíos de su implementación y las métricas de su rendimiento. Esta sección definirá los conceptos clave que todo desarrollador en este campo debe conocer.

- > Tarea
- > Comunicación
- > Sincronización
- > Balance de carga
- > Balance dinámico
- > Escalabilidad
- > Overhead
- > Speed-up
- > Condición de carrera
- > Exclusión mutua

Profundicemos en el concepto de *speed-up*. Un ***speed-up* ideal**, o lineal, ocurre cuando al usar P procesadores el programa se ejecuta P veces más rápido. En la práctica, esto es raramente alcanzable debido al *overhead* de la comunicación y la sincronización.



Veamos estos conceptos en un ejemplo.

Si un programa secuencial tarda 120 segundos en ejecutarse. Al paralelizarlo en 8 procesadores, el tiempo de ejecución se reduce a 25 segundos. Entonces, ¿cuál es el *speed-up* obtenido?

Si el $S_p = T_s/T_p$ entonces, $S_p = 120/25 = 4.8$. Notemos que el *speed-up* lineal para 8 procesadores es 8.0. Como el *speed-up* calculado es $4.8 < 8.0$ entonces es un ***speed-up* típico**.

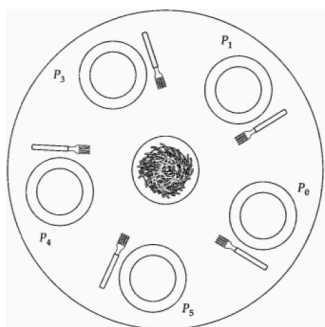
Para ilustrar la importancia crítica de la sincronización, consideremos el problema práctico conocido como "*Too much milk*".

Imaginá que llegás a tu casa y, al ver que no hay leche en la heladera, decidís ir al supermercado. Sin que lo sepas, tu compañero de departamento llega poco tiempo después, ve que no hay leche y también sale a comprar. El resultado: ambos compran leche y ahora hay demasiada.

Este simple problema doméstico es análogo a una condición de carrera en computación: dos tareas (vos y tu compañero) leen un estado compartido (la heladera), toman una decisión basada en esa información y modifican el estado (compran leche), pero lo hacen sin coordinarse.

La **sincronización** (por ejemplo, dejar una nota en la heladera avisando que salimos a comprar leche) es el mecanismo que previene estos resultados incorrectos y garantiza la coherencia de los recursos compartidos. En la programación paralela, este problema se resuelve con mecanismos de sincronización como **locks (cerrojos)** o **semáforos**, que garantizan la exclusión mutua y evitan que múltiples tareas modifiquen un recurso compartido simultáneamente.

Pensemos a partir de otro ejemplo práctico conocido como "*Filósofos cenando*".



Hay cinco filósofos sentados en una mesa redonda. Entre cada filósofo hay un tenedor. Cada filósofo hace dos cosas: pensar y comer. El filósofo piensa por un rato. Cuando tiene hambre agarra el tenedor derecho e izquierdo. No puede comer sin los dos tenedores. Cuando termina de comer los libera. Este ejemplo presenta problemas de competencia, coherencia y seguridad por los recursos y deben usarse mecanismos que eviten dichos problemas.



Veamos un video que aborda este problema y sus posibles soluciones:

The Dining Philosophers Problem

Neso Academy



Ver en

Si necesitás instrucciones para configurar subtítulos en castellano hacé [clic aquí](#).

6. Cierre: diferentes caminos hacia un mismo objetivo



¡Felicitaciones, llegaste hasta el final del módulo 1!

Hemos explorado los conceptos fundamentales que sustentan la computación paralela, desde las arquitecturas, paradigmas y terminología que definen el campo. También hemos visto que no existe una única solución: la elección del camino correcto depende intrínsecamente de la naturaleza del problema a resolver. Para algoritmos con grandes conjuntos de datos y operaciones uniformes, el paralelismo de datos puede ser ideal. Para problemas con funciones independientes o que se pueden descomponer en etapas, el paralelismo de tareas ofrece un modelo más natural. La decisión entre una arquitectura de memoria compartida, con su programación más sencilla y escalabilidad limitada, o una de memoria distribuida, que exige una gestión explícita de la comunicación pero escala a sistemas masivos, también está dictada por los requisitos de la aplicación.

Las CPU *multicore*, las GPU y los clústeres representan tres niveles complementarios del ecosistema del cómputo paralelo. Por ello, una comprensión sólida de la terminología, las arquitecturas subyacentes, los paradigmas de programación y las leyes que gobiernan los límites del rendimiento, es una competencia crucial para cualquier desarrollador, ingeniero o científico que aspire a crear software eficiente y escalable, capaz de explotar todo el potencial del hardware actual y futuro.

En el próximo módulo, procederemos a examinar las arquitecturas de hardware sobre las cuales se implementan y ejecutan estos conceptos.



¡No te vayas sin practicar lo aprendido! Dirígete a la página de **ejercitación** haciendo [clic aquí](#). Recordá que, aunque no es obligatorio que la apruebes, debés ver esta actividad para poder acceder al siguiente módulo.